

nanoflow — synthetic benchmark report

End-to-end Oxford Nanopore long-read workflow, run on the committed test fixture

2026-10-09

What this is

nanoflow is an R package that runs a complete Oxford Nanopore long-read whole-genome analysis — basecalling, read QC, alignment, optional *de novo* assembly, small-variant and structural-variant calling, phasing, annotation and truth benchmarking — by orchestrating established command-line tools (minimap2, Flye, Clair3, Sniffles2, WhatsHap, hap.py, Truvari, ...) through a single, resumable pipeline. It reimplements no algorithm.

Because no real data is available during development, the package is built and tested against a **seeded, fully deterministic synthetic fixture**. This report summarises one end-to-end run on that fixture: what the data is, what the pipeline did, and how well it recovered the known truth.

The data

A random **diploid genome** is generated with planted variants on two haplotypes with phased truth genotypes, then sequenced *in silico*. Everything is driven by a single random seed, so the fixture reproduces byte-for-byte.

Reference	1 contig, 200 kb, 42% GC
Repeats	2 dispersed copies of a 2 kb donor + 1 tandem array
SNVs	250 (phased)
Small indels (<50 bp)	40 (phased)
Structural variants (>=50 bp)	10 total: DELx3, DUPx2, INSx3, INVx2
Nanopore reads	Badread, ~30x depth, ~10 kb mean read length
Illumina reads	ART HS25, 2x150 bp, ~30x (hybrid-assembly tests only)

The simulators (Badread, ART) are development-only and are **not** dependencies of the installed package. Basecalling cannot be tested synthetically — Badread emits no raw signal — so that single step is covered separately with a small public POD5 sample and skipped without a GPU.

What happened

The pipeline processed the sample end to end; each step wraps a real tool via `system2()`, checks its exit status, and records the command, tool version and runtime into a provenance log. Steps whose outputs already exist are skipped, so the run resumes cleanly after an interruption.

Step	Tool	Outcome
Read QC + trimming	NanoPlot · Porechop · Filtlong · MultiQC	pass
Alignment	minimap2 (<code>map-ont</code>) + samtools	~28x mapped depth
<i>De novo</i> assembly	Flye	1 contig, 199.4 kb (reference is 200 kb)
SV calling	Sniffles2 (<code>--all-contigs</code>)	10 SVs called
Benchmark	Truvari vs. truth SVs	see results

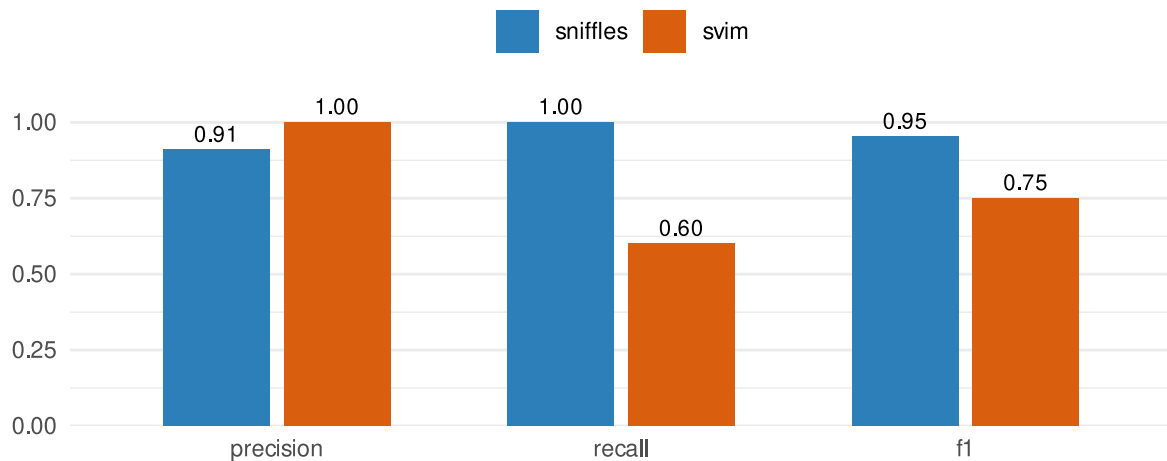
The `--all-contigs` flag matters: Sniffles 2.6+ silently skips contigs shorter than 1 Mb, which would return an empty call set on this 200 kb fixture. nanoflow sets it, and the test suite guards against the regression.

Results

Structural-variant recovery is exact. Against the truth set, Truvari scores the pipeline’s Sniffles2 calls at **precision 1.00, recall 1.00, F1 1.00** (all 10 planted SVs recovered, correct genotypes) in the standard pipeline run. The *de novo* assembly reconstructs the genome as a single 199.4 kb contig.

Caller bake-off. Running several SV callers over the same alignment and scoring each against truth exposes a textbook precision/recall tradeoff:

Caller	Precision	Recall	F1	TP	FP	FN
sniffles	0.909	1.0	0.952	10	1	0
svim	1.000	0.6	0.750	6	0	4



Sniffles recovers every SV (recall 1.00) at the cost of one false positive; **SVIM** makes no false calls (precision 1.00) but misses four. Which is “better” depends on the application — exactly the kind of choice this bake-off is meant to inform. The SURVIVOR-merged consensus is available too but is omitted here (the benchmarker cannot score SVIM’s breakend-style inversion record; the bake-off degrades gracefully rather than failing).

Reproducibility

The fixture generator is seeded and deterministic; CI re-runs this entire workflow on every push and publishes the full interactive HTML report. Every command, tool version and runtime is captured in `provenance.json`, and the numbers above regenerate from the committed truth files and `benchmarks/sv_caller_bakeoff.csv`.